

Profile service

The code for the profile service is given below. The code has been written in Angular.

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs/Observable';
import { FriendProfile } from './friend-add/friend-profile.service';
import { RequestOptions } from '@angular/http';
import { Data } from '@angular/router';

@Injectable()
export class FriendService {
  fprofile: FriendProfile;
  constructor(private http: HttpClient) { }
  getAll(): Observable<any> {
    return this.http.get('/localhost:8083/api/getProfile/');
  }
  addFriend(firstName, lastName, dateOfBirth) {
    this.fprofile = new FriendProfile(firstName, lastName, dateOfBirth);
    const url = '/localhost:8082/api/addProfile/profile';
    this.http.post(url, this.fprofile)
      .subscribe((data: any) => {console.log(data); });
  }
}
```

If you look at the code above, it is pretty simple. It is calling the rest service with a post method passing a JSON object in the body. But, unfortunately, the code was not doing what it was supposed to do—which was to store the data in MongoDB. I was clueless about how to resolve this until I enabled the console and looked at the error.

And then, everything became crystal clear. This is an error due to cross-origin resource sharing (CORS). I knew about it, but it did not strike me then. I immediately went back to the services code and added the CORS annotation to get rid of this error.

```
@RestController
@RequestMapping(value="/api")
@CrossOrigin("")
public class ProfileServiceController {
    @Autowired
    private ProfileService profileService;
    @RequestMapping(method=RequestMethod.POST, value="/addProfile/profile")
```

After solving the problem, I thought of writing this short article on CORS to share my experience with fellow developers.

What is CORS?

To understand CORS, we will need to know what the same-origin policy is. This is a security concept that blocks JavaScript code from making requests to resources in a different domain or origin. CORS, on the other hand, is a mechanism through which a JavaScript running on a Web page can consume a resource running in a different origin.

Cross-origin communication starts with a client making a GET, POST or HEAD request against a server resource. The header of the request contains the origin information of the request. The server in the other domain will inspect the origin information, and will either block or allow the request. If the request is allowed, the server will respond with the requested resource and an 'Access-Control-Allow-Origin' header in the response. The browser allows the request if the origin of the request matches with the 'Access-Control-Allow-Origin' header. The Access-Control-Allow-Origin can be set to a list of allowed requestors or it may also be set to '*'. If it is set to the latter, it means that clients of any origin are allowed, which is not a best practice from the security point of view.

However, if an API is public, it makes sense to set it to '*'. In Spring Boot, we have an easy way to set it up through the CORS annotation, as mentioned earlier. 

By: Rajib Deb

The author leads the open source practice at Infosys. He can be contacted at rajib76@yahoo.com.

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU